

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Dhanush kumar R	Reg. No.:	192411215
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL			100 Marks

SECTION A – CONCEPT MAPPING

Code of Conduct

I, Pharush Kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1	4	4	3	3	4	45
Q2	4	4	3	3	3	43.75
Overall Total (100)						90

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



Name: Dhanush Kumar R

Reg No: 19241215

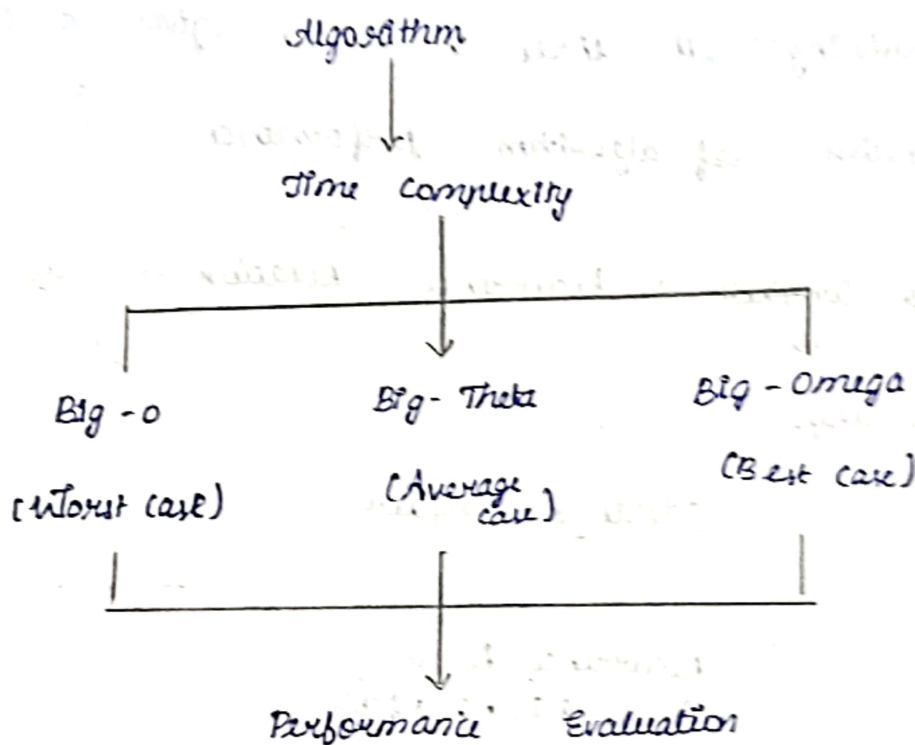
CO1 - AT2

course code: CSA0618

course Name: Design and analysis of algorithm

1) Algorithm $\rightarrow$ Time Complexity $\rightarrow$ Asymptotic Notation

Concept Map



Explanation:

* Algorithm: A step by step procedure to solve a problem

* Time complexity: Measures how the running time changes as the input size (n) increases

Exp.
* Asymptotic Notation: Expresses algorithm efficiency for large input sizes

i) Big-O (O): Represents the worst case performance

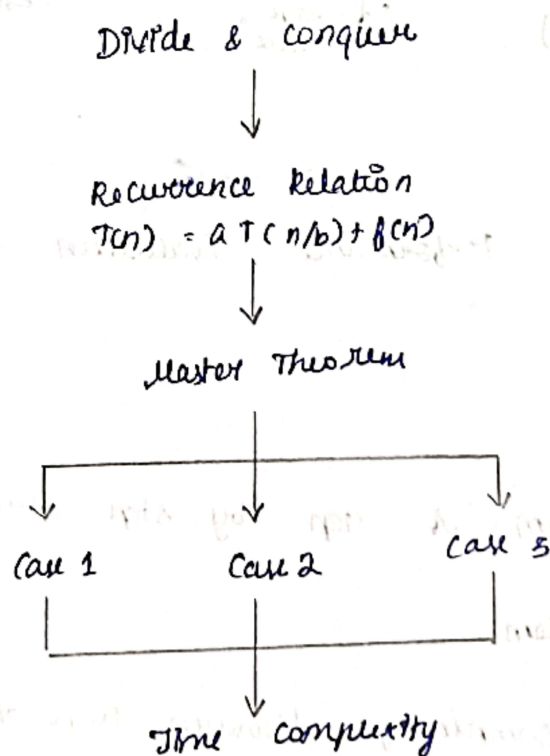
ii) Big-Theta (Θ): Represents the average or exact growth rate

iii) Big-Omega (Ω): Represents the best case performance

* Combining all three notations gives a complete evaluation of algorithm performance.

2) Divide & Conquer $\rightarrow$ Recurrence Relation $\rightarrow$ Master Theorem

concept map

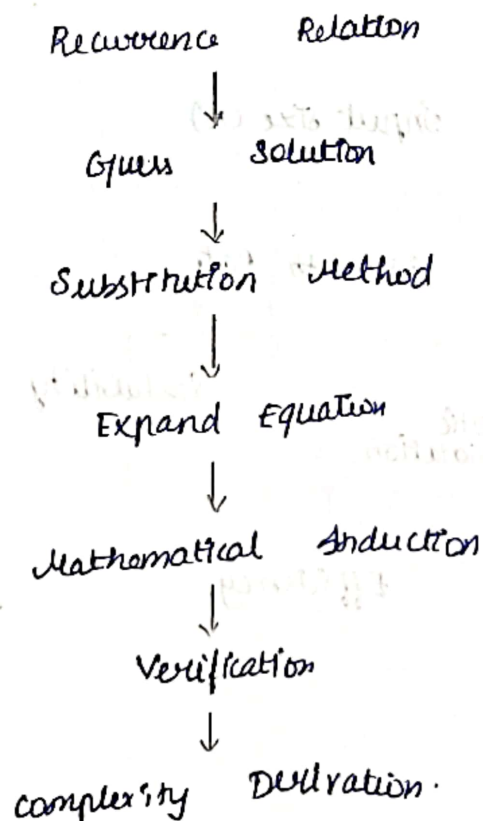


Explanation :-

- * **Divide & Conquer**: Breaks a problem into smaller subproblems.
- * **Recurrence Relation**: Represents the running time of recursive algorithms.
- * **Master Theorem**: Solves recurrence relations directly.
 - i) Case 1: Recursive work dominates.
 - ii) Case 2: Recursive work and dividing work are balanced.
 - iii) Case 3: Dividing / combining work dominates.
- * The result is the algorithm's final time complexity.

3) Recurrence Relation $\rightarrow$ Substitution Method $\rightarrow$ complexity Derivation

concept map:-

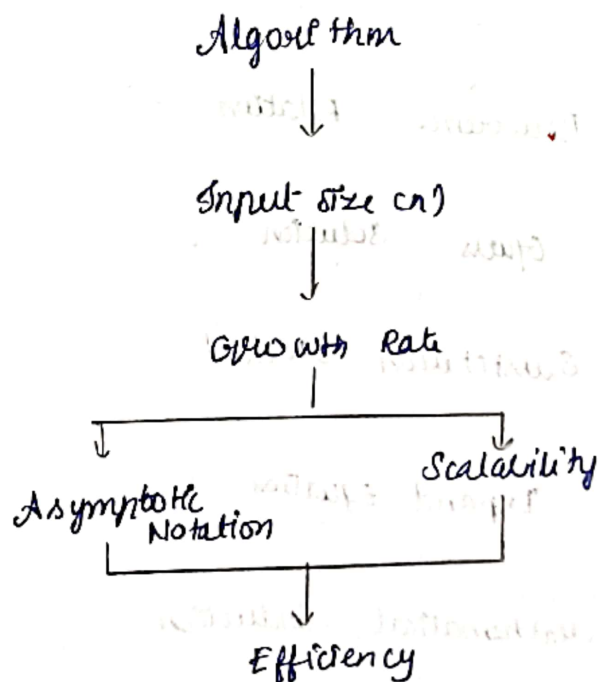


Explanation :-

- * Start with a Recurrence Relation
- * Guess the expected solution
- * Use the Substitution repeatedly
- * Apply Mathematical Induction to prove the guessed solution
- * Verify that the solution satisfies the recurrence
- * The final result is the algorithm's Time Complexity.

4) Algorithm $\rightarrow$ Input size $\rightarrow$ Growth Rate $\rightarrow$ Efficiency

Concept Map :-

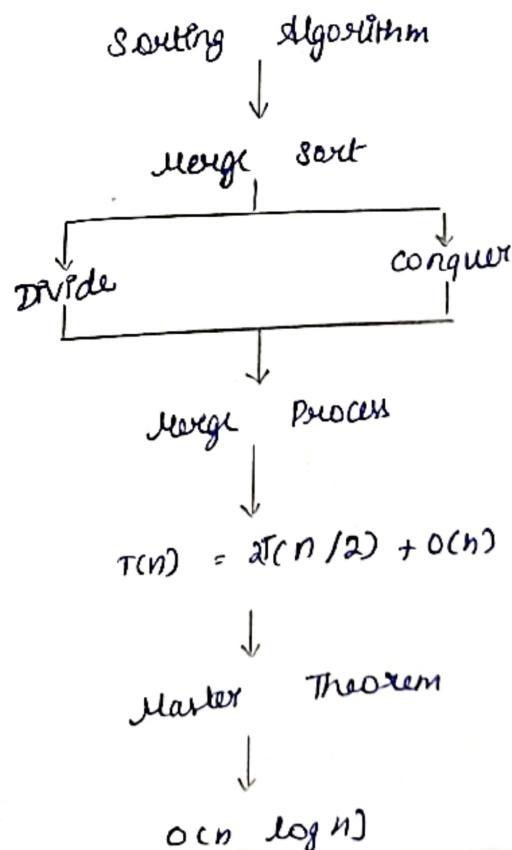


Explanation:

- * **Algorithm**: Solves a computational problem
- * **Input size (n)**: Number of input elements
- * **Growth Rate**: Shows how execution time increases with input size
- * **Asymptotic Notation**: Describes growth mathematically
- * **Scalability**: Measures how well the algorithm performs for large inputs
- * **Efficient algorithms** have slower growth rates and better scalability

5) Sorting Algorithm $\rightarrow$ Merge Sort $\rightarrow O(n \log n)$

concept map



Explanation:

- * **Sorting Algorithm:** Arranges data in ascending or descending order
- * **Merge Sort:** uses the divide and conquer technique
- * **Divide:** Splits the array into two equal halves
- * **Conquer:** Recursively sorts each half
- * **Merge Process:** Combines the sorted halves into one sorted array.
- * The recurrence relation is $T(n) = 2T(n/2) + O(n)$
- * Applying the Master Theorem gives the final time complexity $O(n \log n)$
- * Merge sort is efficient because it guarantees $O(n \log n)$ performance even in the worst case